

```

def is_safe(board, row, col, n):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, n, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True

def solve_n_queens(board, col, n):
    # Base case: If all queens are placed
    if col >= n:
        return True

    # Try placing this queen in all rows of the current column
    for i in range(n):
        if is_safe(board, i, col, n):
            board[i][col] = 1 # Place queen

            # Recur to place the rest of the queens
            if solve_n_queens(board, col + 1, n):
                return True

            # If placing queen here doesn't lead to a solution, backtrack
            board[i][col] = 0

    return False

def print_board(board):
    for row in board:
        print(" ".join("Q" if x == 1 else "." for x in row))

# Get input from user
try:
    N = int(input("Enter the value of N: "))
    if N < 1:
        print("Please enter a positive integer.")
    elif N in [2, 3]:
        print(f"No solution exists for N = {N}")
    else:
        # Initialize an NxN board with 0s
        chess_board = [[0 for _ in range(N)] for _ in range(N)]

        if solve_n_queens(chess_board, 0, N):
            print(f"\nOne of the solutions for {N}-Queens:")
            print_board(chess_board)
        else:
            print("No solution exists.")
except ValueError:
    print("Invalid input. Please enter an integer.")

```

Enter the value of N: 9

One of the solutions for 9-Queens:

```

Q . . . . . . . .
. . . . Q . . . .
. Q . . . . . . .
. . . . . Q . . .
. . . . . . . Q .
. . Q . . . . . .
. . . . . . Q . .
. . . Q . . . . .
. . . . . . . Q .

```

Start coding or [generate](#) with AI.

